



Prof. Iaçanã Ianiski Weber

Exercícios adaptados de Paar, Pelzl e Güneysu, *Understanding Cryptography*, 2ª ed., Springer, 2024 (Capítulo 14 — *Key Management*).

Observação: as soluções dos exercícios *ímpares* seguem o manual de soluções dos autores; em itens discursivos, respostas equivalentes podem ser igualmente corretas.

Parte I — Distribuição de Chave Simétrica (KDC e Kerberos)

Exercício 1

Uma chave-mestra k_{MK} é trocada de forma segura entre as partes (p.ex. via DHKE com certificados). Em seguida, chaves de sessão são geradas regularmente por uma **KDF**. Considere três métodos ($h(\cdot)$ é uma função hash segura e k_i é a i -ésima chave de sessão): (Livro, Problema 14.1)

$$(1) \quad k_0 = k_{MK}, \quad k_{i+1} = k_i + 1 \qquad (2) \quad k_0 = h(k_{MK}), \quad k_{i+1} = h(k_i)$$

$$(3) \quad k_0 = h(k_{MK}), \quad k_{i+1} = h(k_{MK} \parallel i \parallel k_i)$$

- (a) Quais são as principais diferenças entre os três métodos?
- (b) Qual(is) método(s) oferece(m) PFS?
- (c) Se Oscar obtém a n -ésima chave de sessão, quais sessões ele decifra em cada método?
- (d) Qual método permanece seguro se k_{MK} for comprometida?

(a) (1) a chave de sessão é derivada por uma operação **linear e invertível** da chave anterior; (2) usa função hash, gerando uma correlação **não-linear** entre as chaves; (3) usa **a chave-mestra e a chave anterior** em cada derivação.

(b) Os métodos **(2) e (3)**, pois as chaves de sessão antigas não podem ser extraídas da chave recente (o hash é de mão única).

(c) (1) **todas** as sessões (não há PFS — a relação é linear e invertível); (2) a sessão n e **todas as seguintes** (pode-se iterar o hash para frente, mas não para trás); (3) **apenas a sessão atual** (cada derivação exige a chave-mestra, desconhecida).

(d) **Nenhum:** se k_{MK} vazar, *todas* as chaves de sessão podem ser recalculadas (em (1), $k_0 = k_{MK}$; em (2) e (3), $k_0 = h(k_{MK})$ e a recorrência segue a partir daí).

Exercício 2

Rede ponto-a-ponto com **1000 usuários** que se comunicam de forma autenticada e confidencial. (Livro, Problema 14.2)

- (a) Quantas chaves ao todo com **simétricos** e **sem** KDC (chave distinta por par)?
- (b) E com um **KDC**?
- (c) Qual a principal vantagem do KDC?
- (d) Quantas chaves com algoritmos **assimétricos**?

- (a) Cada par precisa de uma chave própria: $\binom{1000}{2} = \frac{1000 \cdot 999}{2} = \underline{\underline{499\,500}}$ chaves.
- (b) Cada usuário compartilha apenas *uma* KEK com o KDC: 1000 chaves de longo prazo.
- (c) O número de chaves de longo prazo cai de $\approx n^2/2$ para n ; um novo usuário precisa de um único canal seguro (com o KDC), em vez de um com cada um dos demais. (Contrapartida: o KDC vira ponto único de falha e gargalo.)
- (d) Cada usuário possui *um par* de chaves: 1000 pares (2000 chaves — 1000 públicas + 1000 privadas), sem chave por par. Porém, as chaves públicas precisam ser **autenticadas** (certificados/PKI).

Exercício 3

Escolha das cifras de um **KDC** com duas classes de cifragem: $e_{k_{U,KDC}}(\cdot)$ (KEK de longo prazo) e $e_{k_{ses}}(\cdot)$ (sessão). Disponíveis: **PRESENT-80** e **AES-256**, uma para cada classe. Qual para qual? Justifique. (Livro, Problema 14.3)

AES-256 para a KEK (KDC $\leftrightarrow$ usuário, longo prazo) e **PRESENT para a sessão**.

Justificativa: PRESENT-80 não oferece boa segurança de *longo prazo* (embora hoje resista à força bruta de adversários não-estatais). Se uma chave de sessão for quebrada — algo provável quando houver computadores quânticos —, apenas *aquela* sessão é comprometida. Já o AES-256 oferece excelente segurança de longo prazo (mesmo contra computadores quânticos) e deve proteger a KEK: se uma $k_{U,KDC}$ for descoberta, *toda* a comunicação passada e futura do usuário U ficaria exposta — exige-se segurança muito mais forte aqui.

Exercício 4

Mostre que o **PFS não é garantido** no Kerberos simplificado: como Oscar decifra comunicações passadas e futuras se (a) k_A é comprometida; (b) k_B é comprometida. (Livro, Problema 14.7)

- (a) Comprometida a KEK k_A de Alice, Oscar calcula a chave de sessão k_{ses} (transmitida cifrada sob k_A , em $y_A = e_{k_A}(k_{ses}, \dots)$) e, assim, **decifra todas as mensagens** — passadas (se armazenou os criptogramas) e futuras.

(b) O mesmo vale para a KEK k_B de Bob.

Como as chaves de sessão são protegidas *pelas* KEKs de longo prazo, o esquema não oferece PFS.

Exercício 5

Pay-TV: DH com módulo de 2048 bits, mas só **DES** para os dados; derivação $K^{(i)} = f(K_{AB} \| i)$. (Livro, Problema 14.9)

- (a) Quantos GB para um filme de 2 h a 1 Mbit/s? É realista armazenar?
- (b) Se o atacante quebra uma chave DES em 10 min, com que frequência derivar a chave para impedir a decifragem *offline* de um filme de 2 h em menos de 30 dias?

(a) Armazenamento = taxa $\times$ tempo = 10^6 bit/s $\times$ 2 h = $10^6 \times 2 \times 3600 = 7,2 \times 10^9$ bits = 7,2 Gbit = 0,9 GB. Armazenar $\approx$ 0,9 GB é trivial (cabe num pen drive barato) — realista.

(b) Chaves recuperáveis em 30 dias: $\frac{30 \text{ dias}}{10 \text{ min}} = \frac{30 \cdot 24 \cdot 60}{10} = 4320$. Para impedir a decifragem do filme todo nesse prazo, cada chave deve cobrir no máximo $T = \frac{2 \text{ h}}{4320} \approx$ 1,67 s de vídeo. Logo, deriva-se uma nova chave a cada $\approx$ 1,67 s. Como funções hash são rápidas, essa taxa é facilmente atendível em software.

Parte II — Man-in-the-Middle, Certificados e PKI

Exercício 6

DHKE com MITM: $p = 467$, $\alpha = 2$, $a = 228$ (Alice), $b = 57$ (Bob), $o = 16$ (Oscar). (Livro, Problema 14.11)

- (a) Calcule k_{AO} e k_{BO} (i) como Oscar os calcula e (ii) como Alice e Bob os calculam.
- (b) O que isso mostra sobre a necessidade de autenticar as chaves públicas?

Oscar injeta $\tilde{O} = \alpha^o = 2^{16} \equiv 156 \pmod{467}$ no lugar das chaves públicas reais.

(a) Alice: $A = 2^{228} \equiv 394$; $k_{AO} = \tilde{O}^a = 156^{228} \equiv$ 243 (mod 467).

Bob: $B = 2^{57} \equiv 313$; $k_{BO} = \tilde{O}^b = 156^{57} \equiv$ 438 (mod 467).

Oscar: $O = 2^{16} \equiv 156$; $k_{AO} = A^o = 394^{16} \equiv$ 243; $k_{BO} = B^o = 313^{16} \equiv$ 438 (mod 467).

(b) Os valores coincidem (Oscar e Alice obtêm 243; Oscar e Bob obtêm 438), mas Alice e Bob não compartilham chave comum *entre si* — cada um tem uma chave com Oscar. Como as chaves públicas não foram autenticadas, o ataque passa despercebido: a **autenticação das chaves públicas** (via certificados) é imprescindível.

Exercício 7

DHKE com certificados: ataque de substituição. (Livro, Problema 14.13)

- (a) Oscar troca $\text{Cert}(B)$ por um $\text{Cert}(O)$ válido (dele). Como é detectado?
- (b) Oscar troca a chave pública b_B por b_O (mantendo ID_B). Como é detectado?

(a) Alice detecta porque $\text{Cert}(O)$ contém o **ID de Oscar**, e não $ID(B)$. Ao ler o certificado, ela percebe imediatamente que aquele não é o certificado de Bob.

(b) Detectado quando Alice **verifica a assinatura** do certificado com $k_{pub,CA}$. Como o conteúdo assinado (a tupla chave pública + ID) foi alterado, a verificação da assinatura **falha**. (É justamente isso que os certificados garantem: *ligar* a identidade à chave pública de forma resistente à adulteração.)

Exercício 8

Chaves públicas DH certificadas por uma CA. Oscar obtém a chave **privada** de assinatura da CA. Ele decifra mensagens antigas armazenadas? Se não, que outro ataque passa a poder fazer? (Livro, Problema 14.15)

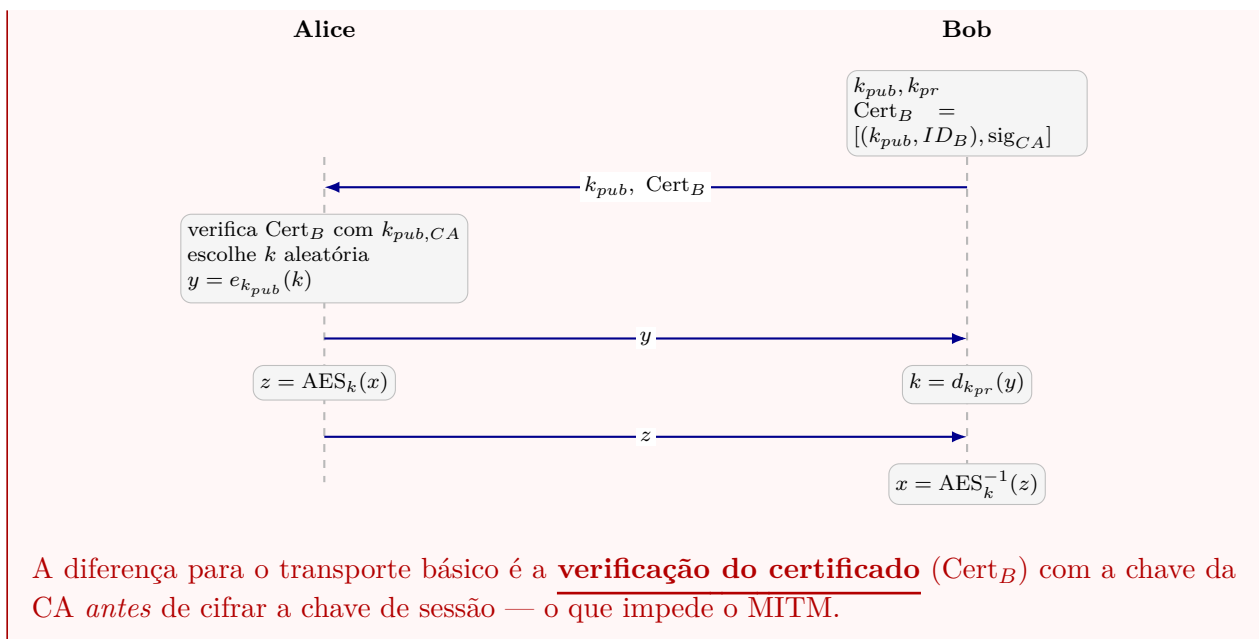
A assinatura da CA cobre apenas a *chave pública* do usuário, $k_{pub,U} = \alpha^{a_U}$. Mesmo com a chave privada da CA, Oscar **não** consegue calcular a chave *privada* de nenhum usuário — isso exigiria resolver o **problema do logaritmo discreto**. Logo, ele também não calcula as chaves de sessão usadas *antes* do comprometimento e **não decifra** os criptogramas antigos.

Contudo, dali em diante Oscar pode **se passar por qualquer usuário e gerar certificados falsos** (assinando chaves públicas que ele controla), viabilizando ataques MITM futuros. Ou seja: o comprometimento da chave da CA não quebra a confidencialidade *passada*, mas permite **personificação** a partir de então.

Exercício 9

Desenhe um protocolo de **transporte de chave** com **RSA e certificado**. (Livro, Problema 14.17)

Bob possui k_{pub}, k_{pr} e $\text{Cert}_B = [(k_{pub}, ID_B), \text{sig}_{CA}]$.



Exercício 10

PGP: descreva o modelo de confiança e o gerenciamento de chaves; aponte vantagem e desvantagem frente às CAs hierárquicas. (Livro, Problema 14.19)

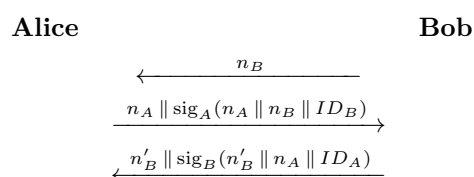
O PGP usa a **Teia de Confiança** (*Web of Trust*). Diferentemente das arquiteturas em árvore (hierárquicas), a teia consiste apenas em **nós (usuários) com direitos iguais**, em que cada nó pode **certificar** outros. A validade de um certificado vem de uma **cadeia de confiança**: Alice confia nos certificados recebidos de alguém que conhece (p. ex. Bob) — idealmente verificando a *impressão digital* (*fingerprint*) com ele — e, por transitividade, também confia nos certificados em que Bob confia, e assim por diante.

Vantagem: dispensa uma CA mutuamente confiável. **Desvantagem:** é preciso garantir que todos os usuários sejam honestos e não aceitem certificados falsos.

Desafio — Autenticação Mútua

Exercício 11

Protocolo de autenticação mútua por assinaturas (n_X é um *nonce* de X): (Livro, Problema 14.21)



(a) Descreva os passos e o objetivo de cada um.

(b) Explique o **ataque de intercalação** (*interleaving*) e suas consequências.

(c) Contraste-o com o **MITM**.

(a) *Passos:*

- Bob envia o nonce n_B a Alice (usado adiante para evitar *replay*).
- Alice assina n_B junto com seu próprio nonce n_A e a identidade ID_B , e envia a assinatura com n_A a Bob.
- Bob verifica a assinatura com a chave pública de Alice. Se confere, ele sabe: (i) que não é *replay*, pois Alice assinou o nonce *atual* n_B ; e (ii) que é de fato Alice se autenticando *para Bob*, pois ID_B foi assinado. Para confirmar, Bob assina n_A com um novo nonce n'_B e a identidade ID_A .
- Alice verifica com a chave pública de Bob: aprende que vem de Bob (assinatura confere), que não é *replay* (contém seu nonce atual n_A) e que Bob quer se autenticar *para Alice* (pois ID_A foi assinado).

(b) Oscar apenas repassa as mensagens que recebe das duas partes (mantendo, em paralelo, uma sessão com Bob e outra com Alice). Consequência: Oscar personifica Alice perante Bob e personifica Bob perante Alice. É difícil de evitar porque Oscar *não* manipula segredos criptográficos (chaves) — apenas se interpõe numa troca legítima de mensagens.

(c) O *interleaving* atinge **protocolos de autenticação** e viola a **autenticação de entidade**. Já o **MITM** ataca a **troca de chave** e leva ao estabelecimento incorreto de chaves. As consequências do MITM tendem a ser mais abrangentes: chaves incorretas (conhecidas pelo atacante) permitem quebrar vários serviços de segurança (confidencialidade, integridade etc.).